Modern Cloud Computing

Lecture 19: Modern Resource Management II

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Serverless recap
- **YuanRong** (Huawei, SIGCOMM 2024): making serverless *general-purpose*
  - Fast function system, built-in data system, Service Bridge
- **DROPS** (Microsoft Azure Functions, EuroSys 2026): sizing pools of pre-warmed containers
  - Workload characterization, why forecasting fails, a statistical demand–supply method

Serverless Clouds

Users:

- Write **functions**, triggered by events (HTTP, queue, timer)
- No servers to manage; **automatic scaling** (including scaling to zero) handled by provider
- **Pay per use**: billed for number of requests
- State lives in **Backend-as-a-Service** (object storage, DBs, queues)

Providers

- Turn an invocation into a running **container/ microVM** in milliseconds
- Avoid **cold starts** (usually by keeping resources *pre-warmed*)
- Isolate tenants; route calls; scale thousands of instances

Two questions for today:

1. *Scope*: today's FaaS is mostly simple stateless handlers. Can serverless run **general distributed applications**? → **YuanRong**
2. *Economics*: warm pools hide cold starts but cost money. **How big should they be?** → **DROPS**

Part 1: YuanRong

A Production General-purpose Serverless System for Distributed Applications

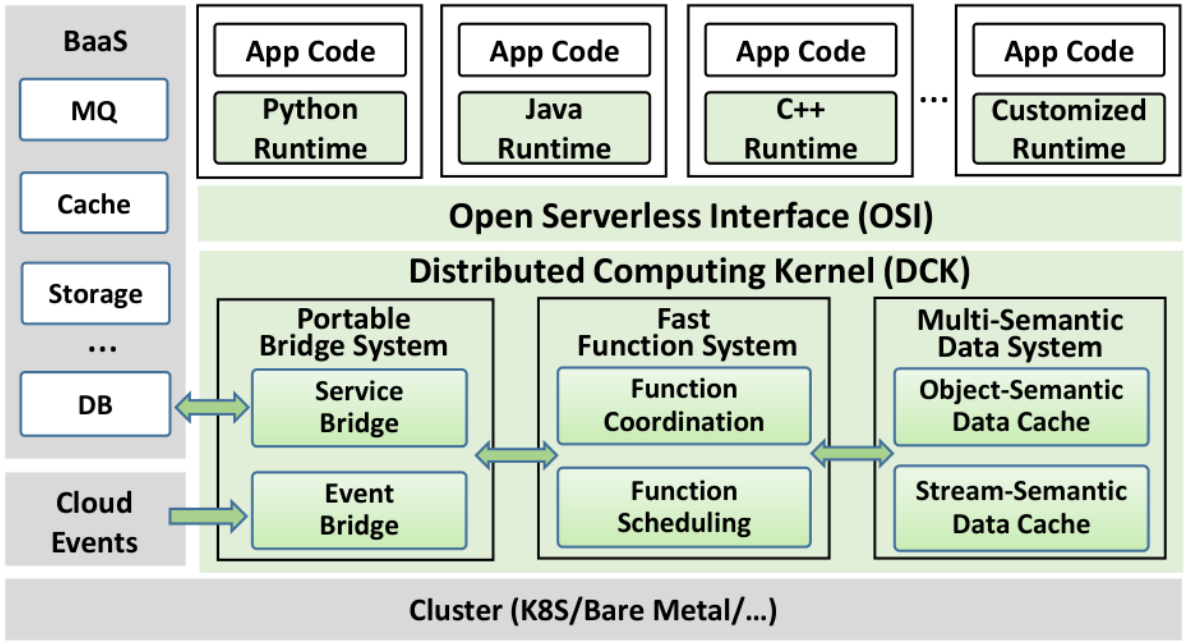
Chen et al., Huawei — SIGCOMM 2024

Why Today's FaaS Can't Run General-Purpose Applications

Existing FaaS is dominated by simple **event-driven request handlers**: an instance serves one request and rarely talks to other instances. Video processing, analytics, ML, and microservices need more.

	Limitation
L1	No **fine-grained coordination**: functions are unaddressable; coordination goes through external storage or queues
L2	**State and data transfer via remote storage**: hundreds of ms of latency, limited bandwidth
L3	Poor **stateless** ↔️ **stateful** interaction
L4	No **unified API** across backend services

YuanRong at a Glance



(Source: Chen et al., SIGCOMM 2024, Fig. 1)

- **Open Serverless Interface (OSI)**: one API implemented in C/C++, with runtime support for other languages
- **Distributed Computing Kernel (DCK)** with three subsystems:
  - **Fast function system** → L1
  - **Multi-semantic data system** → L2
  - **Portable bridge system** → L3, L4
- Runs on Kubernetes or bare metal
- In production **3+ years**, **~20 regions**, up to **30B requests/day** on **>100,000 cores**, **53%** average daily CPU usage

Programming Model: Functions, Futures, Objects, Streams, Bridges

Category	Key OSI calls
Function	Create, Invoke (async), Exit, Kill, SaveState, LoadState
Object	Put, Get, GInc / GDec (reference counts)
Stream	Publish, Subscribe, Send, Flush, Recv, Ack
Service Bridge	CreateBridge; Execute / Commit / Rollback (SQL); Set / Get / Expire (cache); PutObj / GetObj (object store)
Event Bridge	CreateChanel, AddEvent, AddTarget

(Condensed from Chen et al., SIGCOMM 2024, Table 1)

```python
@yr.invoke
def add_one(x):
    return x + 1

# resource hint: 1 core, 1 GB
objref = add_one.options(cpu=1000, memory=1024).invoke(5)
print(yr.get(objref))   # 6
```

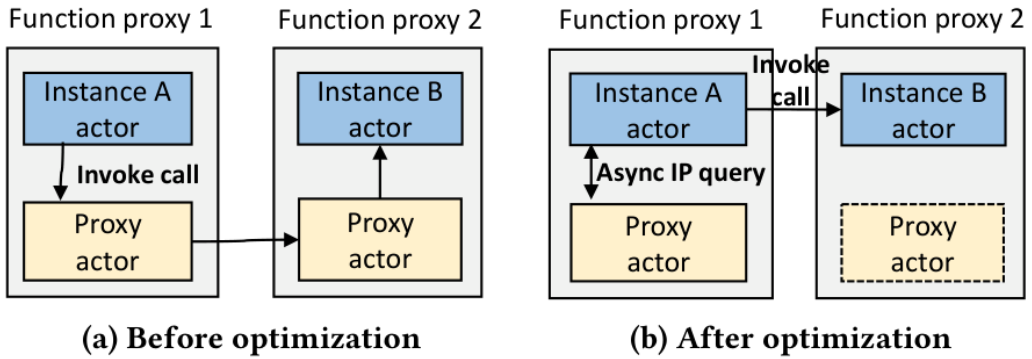
(Python example adapted from Fig. 2)

- `invoke` is **asynchronous** and returns a **future** stored in the data system
- Arguments can be passed **by reference**: data is pulled only when the callee needs it
- Same API from Python, Java, and C++

Fast Function System: Sub-millisecond Invocation

- A **function proxy** on every node forwards messages (actor-style, async RPC)
- **Before**: caller actor → local proxy actor → remote proxy actor → callee: several async hops per call
- **After**: the caller asynchronously **queries the callee's routing info once**, then sends invocations **directly** to the callee instance
- Combined with pass-by-reference, the invocation message stays tiny

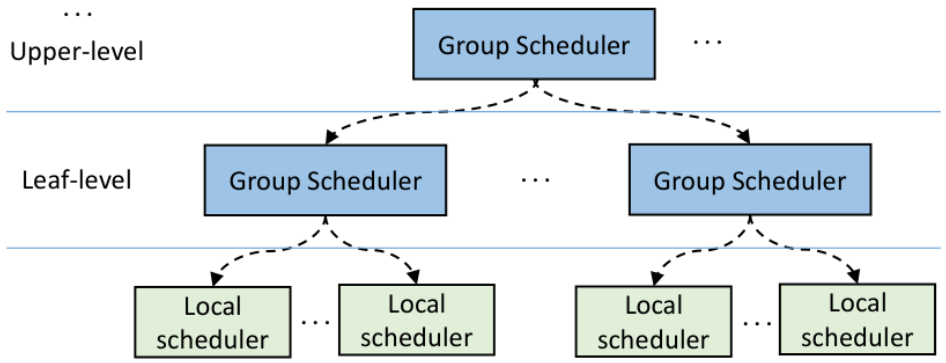
Result: ~1 ms local calls, <2 ms cross-node with 100 KB payloads; Alibaba FC and AWS Lambda are **10–40× slower** (they route through extra services)



(Source: Chen et al., SIGCOMM 2024, Fig. 3)

Fast Function System: Hierarchical, Locality-aware Scheduling

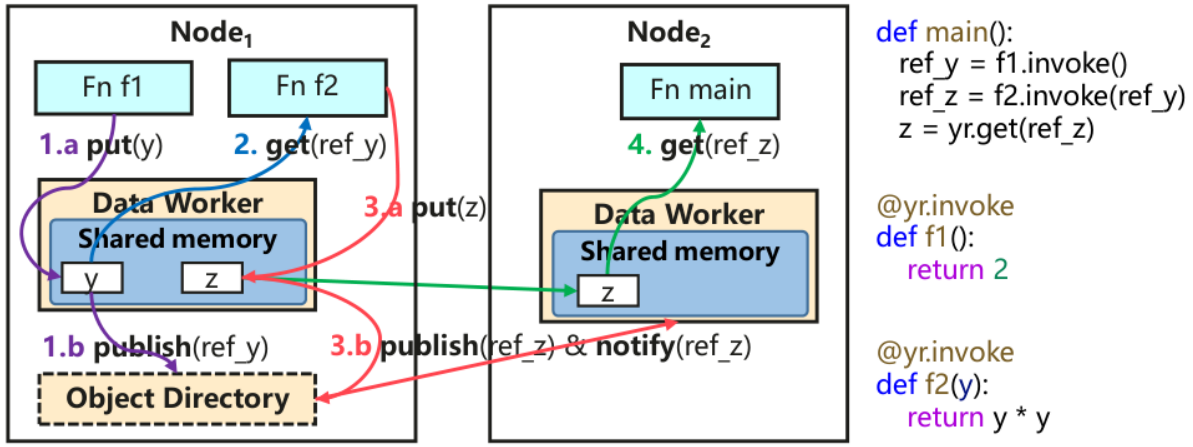
- **Local scheduler** per node; **group schedulers** form a tree above them
- **Top-down** (request from the API gateway): pick the least-busy group, descend to a node
- **Bottom-up** (a function creates a callee): try the **caller's own node** first, then its leaf group, then higher (keeps call chains and their data close)
- Sync schedulers' views:
  - **Pull**: group schedulers pull views every 1 s
  - **Push**: local schedulers check every 100 ms and push big changes
  - **Revise**: a failed placement returns a corrected view
- If a group scheduler is congested, local schedulers ask the root to switch them to another parent



(Source: Chen et al., SIGCOMM 2024, Fig. 4)

Multi-Semantic Data System: Object Cache

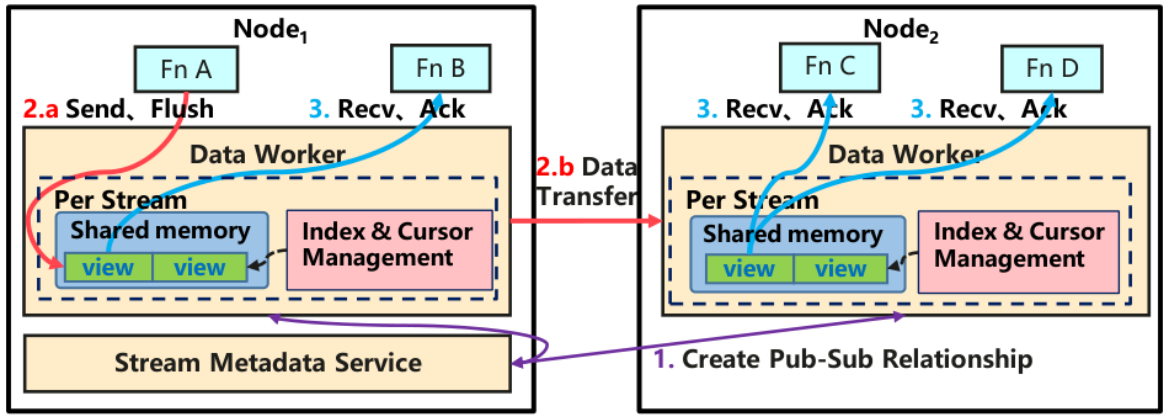
- Insight: FaaS nodes have **under-used memory** → pool it into a distributed cache
- A **data worker** per node: functions put/get through **local shared memory**; workers move and cache copies across nodes
- An **object directory** maps object IDs to locations and notifies waiters when an unresolved future becomes visible
- Mutable or immutable objects; **PRAM** or **causal** consistency; last-writer-wins by default, optional global locks
- Cold copies are evicted; primary copies spill to local SSD
- **~200 µs** object exchange: orders of magnitude faster than Redis or object storage



`main` calls f1 then f2 with f1's result by reference (Source: Fig. 5)

Multi-Semantic Data System: Streams

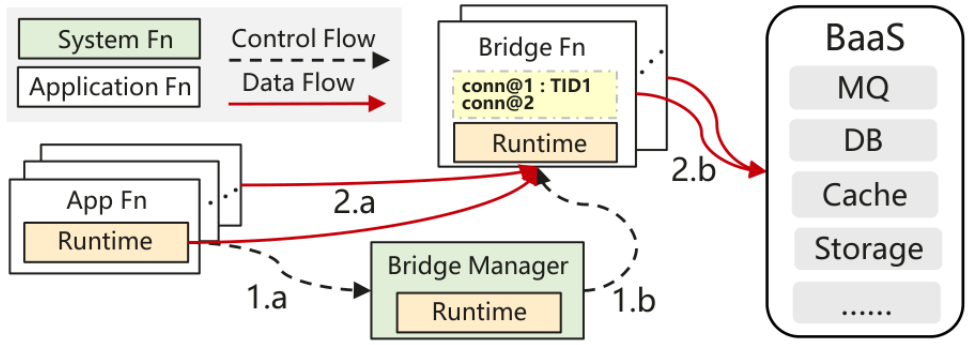
- **Pub-sub streams** for unbounded data: one-to-many (broadcast) and many-to-one (gather/reduce)
- The relationship is set up **once**; sending and receiving need no further metadata operations
- **Page-based** shared memory: producers append elements to pages, Flush makes them visible, pages are pushed to remote consumers asynchronously
- Same-node consumers read the **same pages** (zero copy); a remote node receives each page once
- **~2 ms** end-to-end per element at **5 Gbps**, instead of a function call or queue service per batch



(Source: Chen et al., SIGCOMM 2024, Fig. 6)

Portable Service Bridge: Stateless Functions, Stateful Backends

- **Bridge functions** are themselves serverless functions, each holding a **pool of BaaS connections**; a **bridge manager** creates, reuses, scales, and restores them
- **Unified API** → multi-cloud portability (L4)
- **Connection reuse** (L3): many user functions share a few authenticated connections
- **Distributed → local transactions**: functions in one transaction send their SQL through the same bridge instance by transaction ID



(Source: Chen et al., SIGCOMM 2024, Fig. 7)

Migrating Microservices to Serverless: Four Challenges

Before migration, these internal microservices averaged **~5% CPU utilization**.

Challenge	YuanRong's fix
1. Customers won't make significant changes to their code	Custom framework and keep interface the same
2. Whole-app functions barely improve utilization (the app still holds its minimum reservation)	Profile call frequency per job; **split hot jobs into their own functions**, bundle cold jobs together
3. Splitting functions causes DB transactions inconsistency	Route them through a **Service Bridge** instance → local transaction
4. Cold starts make scale-to-zero impractical	**Predict** periodic hot functions a day ahead and pre-create instances; **QuickStart** process snapshots (CRIU) for aperiodic ones

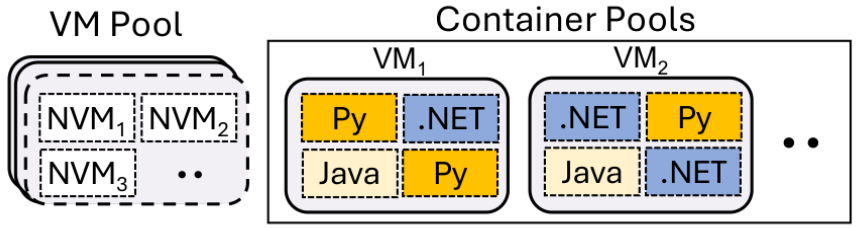
Part 2: DROPS

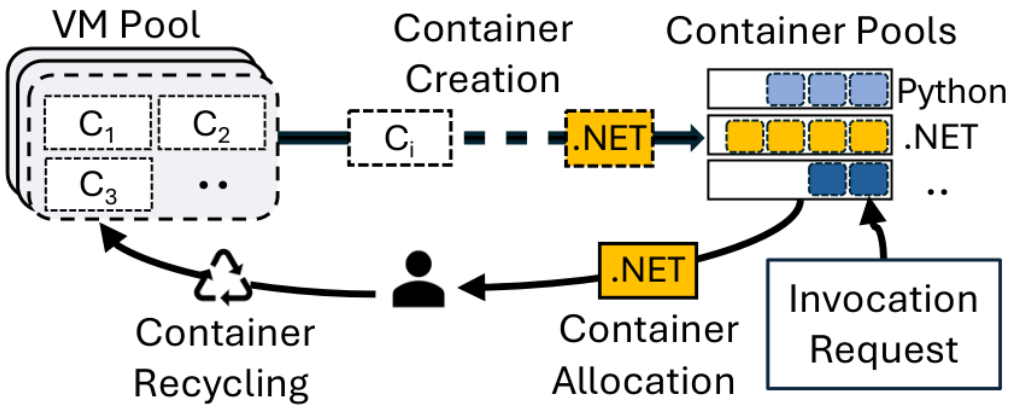
Managing Serverless Resource Pools in Microsoft Azure Functions

Alquraan et al., Univ. of Waterloo + Microsoft — EuroSys 2026

Azure Functions

- Platform = **VMs**, each hosting **nested VMs (NVMs)**, each running **one container** (tenant isolation)
- **No oversubscription**: each NVM reserves exclusive cores and memory
- A **pre-warmed container** has the runtime (e.g., Python 3.11, 1 core) loaded but **no user code**
- On first invocation: take one from the pool, inject user code → serve → **deleted when idle**; cores go back to the VM





(Source: Alquraan et al., EuroSys 2026, Figs. 1 and 3)

- One **container pool per runtime/size** (e.g., ".NET 8.0/1") plus one **VM pool**
- Every consumption triggers **replenishment** back to a target size

The Pool-Sizing Problem

Two kinds of failure when a request can't be served from the pool:

Failure	What happens	Latency
Container-allocation failure	No pre-warmed container, but a VM has free cores → create a container	100s of ms – a few s
Core-allocation failure	No VM has free cores → provision a new VM	**minutes**

- **SLO** = target **success rate**: fraction of allocation requests served immediately from a warm pool
- Too small → failures and SLO violations; too large → idle resources
- Operators **oversize** to be safe: before DROPS, warm pools were **76% of the platform's total cost**

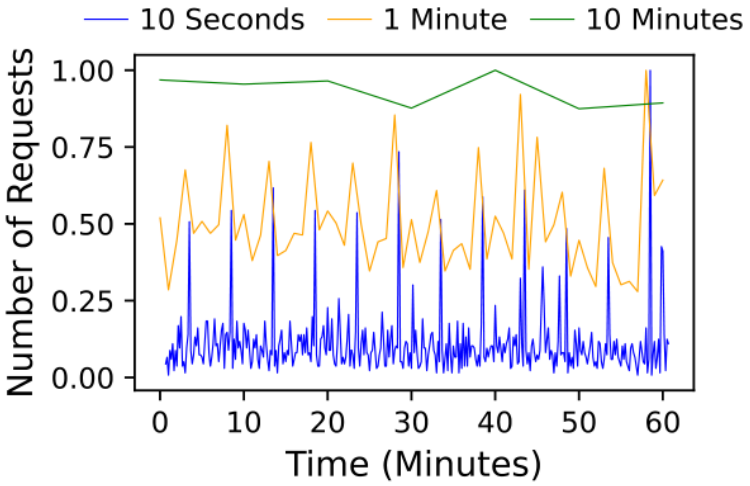
Can We Just Forecast the Load?

Statistical (Naive, ETS, Theta, ARIMA), ML (DeepAR, TFT, PatchTST), foundation model (Chronos), and an ensemble — via AutoGluon, 7-day training, rolling 1-day tests on .NET 8.0/1:

Prediction interval	Best avg. error (MAPE)	Max errors
10 seconds	84% (DeepAR)	889% – 3,630%
1 minute	30% (Ensemble)	2,326% – 5,404%
10 minutes	19%	579% – 901%
1 hour	**14%** (ARIMA)	74% – 358%

(Condensed from Alquraan et al., EuroSys 2026, Table 2)

- Bursts specifically: avg. error **107%**, max **1,170%**
- ML models take up to ~1 hour to train per day of 10 s forecasts, and are no better



The same hour at 10 s / 1 min / 10 min resolution (Source: Fig. 6)

- Coarse intervals are easier to predict only because they **smooth the bursts away** (coefficient of variation 1.88 → 0.32 → 0.05)
- But the pool has to survive the bursts, second by second

Classic Buffer-Sizing Approaches

Static (Azure before DROPS)

- Fixed pool size per pool, set by experts; VM pool by low/high **watermarks** on idle cores
- ✓ Simple, no runtime logic
- ✕ Hand-tuned for many pools; oversized "to be safe"

Reactive

- Grow/shrink the target size dynamically
- ✓ Adapts automatically
- ✕ **Lagging**: bursts are short and creation is slow — new containers arrive after the burst has failed

Predictive

- Forecast next interval's load, set the pool size in advance
- ✓ Proactive
- ✕ Forecasts miss bursts, and one number per interval must be mapped to a pool size by assuming a distribution

DROPS: The Key Insight

DROPS treats the platform as an **infinite-capacity system**: it can always create more resources (with some **latency**)

- When a request consumes a pool item, a **replenishment** starts immediately
- So a request at time t_r is affected only by requests whose replenishments **haven't finished by** t_r
- Replenishment takes at most the **maximum creation latency** W
- The pool size needed for request r depends **only on demand (and recycling) in the window** $[t_r - W,\ t_r]$

For every historical request, compute the pool size that would have served it, and collect these into a distribution.

The DROPS Algorithm

Algorithm 1 DROPS

Input: Demand trace: R, Recycling trace: Q, Resource creation latency distribution: C, Number of samples: N
Output: Pool-size-to-success-rate mapping: P
Initialize an empty list L
for $i = 1$ to N **do**
  Set WS to $\mathrm{P}_{100}(C)$ ▷ Window size = max creation latency
  for each consumption request r in R **do**
    $PS = 0$ ▷ Start with a pool size of zero
    Set $w \leftarrow [t_r - WS, t_r]$ ▷ Set window w using timestamp of r
    for each request x in w **do** ▷ Analyze requests in w
      $\mathrm{Rep}_{lat} \leftarrow$ sample from C ▷ Latency drawn from C
      if $t_x + \mathrm{Rep}_{lat} > t_r$ **then** ▷ Is replenishment time $> t_r$
        $PS = PS + 1$ ▷ Increment the needed pool size
      end if
    end for
    $q \leftarrow$ count of recycling events with timestamp $\in w$
    $PS = PS - q$ ▷ Resource recycling reduces the needed pool size
    $L \leftarrow L \cup \{PS\}$ ▷ Append pool size PS to L
  end for
end for
Compute P as the empirical CDF of L
return P

(Source: Alquraan et al., EuroSys 2026, Algorithm 1)

For each historical request r:

1. Look at all requests x in the window $[t_r - W,\ t_r]$, where W = P100 creation latency
2. For each, **sample** a replenishment latency from the empirical distribution
3. If $t_x + \text{latency} > t_r$, that replenishment is still in flight → the pool needs **one more item** at t_r
4. Subtract **recycling** events in the window (items returned to the pool)
5. Record the result PS_r

Repeat N times (sampling), take the **empirical CDF** → a **pool-size-to-success-rate mapping**. For a 99% SLO, use its 99th percentile.

Runs in ~**1 minute** for two weeks (~2M requests); no training.

Example

Creation latency is always **3 s** (so $W = 3$ s), no recycling. Requests arrive at **t = 0, 1, 1.5, 2, 6**.

Request at	Requests in $[t-3,\ t]$	Replenishments still running at t	PS needed
0	0	0 (done at 3)	**1**
1	0, 1	both (done at 3, 4)	**2**
1.5	0, 1, 1.5	all three	**3**
2	0, 1, 1.5, 2	all four	**4**
6	6	itself	**1**

Empirical CDF of {1, 2, 3, 4, 1}: pool size **4 → 100%** success, **3 → 80%**, **2 → 60%**, **1 → 40%**.

The burst at t = 0–2 alone sets the size for a 100% SLO; with random latencies, DROPS samples instead of using the fixed 3 s.

Applying DROPS to Both Kinds of Pool

Container pools

- Demand: that pool's **allocation trace**
- Supply: its **container-creation** latency distribution
- Recycling: **none**

VM pool

- Demand: a synthetic **core-allocation trace**, shifted **earlier** by a sampled container-creation time (cores are taken when the container is built)
- Recycling: **core-recycling trace** — cores return at a time computed by sampling allocation + user + deletion durations
- Supply: **VM-creation** latency distribution

In production (C#, all Azure Functions regions): rerun **every 4 hours** on the **previous week** of traces, plus on demand when failure rates exceed the SLO or utilization looks abnormal. Reported production savings: **80–90%**, varying by region.